

From Statute to Answer: How OpenLeave Works

A walkthrough of the four artifacts in a Rules-as-Code system — the raw rule, the coded rule, the user's facts, and the system's result — using a real determination from the OpenLeave prototype. Every excerpt below is taken from the working code in this repository, and the results were produced by actually running the engine.

Running example: an employee in Minnesota, hired March 2025, working full time at \$1,100/week for an 85-person company, wants bonding leave for a new child starting September 1, 2026.

Stage 1 — Raw rules: what the law says

The authoritative source is natural-language law. Two rules govern our example — one federal, one state.

Federal FMLA — 29 U.S.C. § 2611(2) (excerpt):

(2) Eligible employee. (A) In general. — The term “eligible employee” means an employee who has been employed — (i) for at least 12 months by the employer with respect to whom leave is requested...; and (ii) for at least 1,250 hours of service with such employer during the previous 12-month period.

(B) Exclusions. — The term “eligible employee” does not include — ... (ii) any employee of an employer who is employed at a worksite at which such employer employs less than 50 employees if the total number of employees employed by that employer within 75 miles of that worksite is less than 50.

Minnesota Paid Leave — Minn. Stat. ch. 268B (summarized excerpt):

An applicant is eligible for benefits if the applicant has wage credits of at least 5.3% of the state's average annual wage (§ 268B.04). The weekly benefit amount is 90% of wages that do not exceed 50% of the state's average weekly wage, plus 66% of wages between 50% and 100% of that wage, plus 55% of wages above it, capped at the state average weekly wage (§ 268B.06). Nearly all employers are covered, regardless of size.

Notice what these two rules are made of: **thresholds** (12 months, 1,250 hours, 50 employees), **date arithmetic**, and a **piecewise formula**. This is closed-texture law — exactly the kind that can be encoded faithfully. One phrase in the FMLA (“serious health condition”) is different in kind; we return to it at the end.

Stage 2 — Coded rules: the same law, executable

Each statutory condition becomes a Finding that carries its citation. This is the FMLA eligibility test from [openleave/regimes/fmla.py](https://openleave.org/regimes/fmla.py) — compare it clause-by-clause with the excerpt above:

```
result.findings = [  
    Finding(  
        key="tenure",
```

```

        description="Employed by this employer for at least 12 months",
        met=facts.tenure_months >= 12,
        citation=Citation(f"{USC_2611}(2)(A)(i)"),
        detail=f"Tenure at leave start: {facts.tenure_months:.1f} months",
    ),
    Finding(
        key="hours",
        description=f"At least {min_hours:.0f} hours of service in the previous 12 months",
        met=facts.employee.hours_last_12mo >= min_hours,
        citation=Citation(f"{USC_2611}(2)(A)(ii)"),
        detail=f"Hours in previous 12 months: {facts.employee.hours_last_12mo:.0f}",
    ),
    Finding(
        key="worksite",
        description=f"Employer has {min_headcount:.0f}+ employees within 75 miles of the worksite",
        met=facts.worksite_headcount >= min_headcount,
        citation=Citation(f"{USC_2611}(2)(B)(ii)"),
        detail=f"Employees within 75 miles: {facts.worksite_headcount}",
    ),
]

```

The Minnesota benefit formula from [openleave/regimes/minnesota.py](#) — a direct transcription of § 268B.06's piecewise structure:

```

def _weekly_benefit(aww: float, saww: float) -> float:
    """Progressive replacement per Minn. Stat. § 268B.06: 90% of wages up to
    50% of SAWW, 66% between 50% and 100% of SAWW, 55% above; capped at SAWW."""
    half = 0.5 * saww
    benefit = 0.9 * min(aww, half)
    if aww > half:
        benefit += 0.66 * (min(aww, saww) - half)
    if aww > saww:
        benefit += 0.55 * (aww - saww)
    return round(min(benefit, saww), 2)

```

Numbers that legislatures and agencies revise — wage bases, caps, thresholds — are **not** hard-coded. They live in [openleave/parameters.json](#) as effective-dated series, which is what lets a determination be evaluated under the law as of any date, and what the AI amendment pipeline updates when a rate notice drops:

```

"mn.saww":          [ ["2026-01-01", 1423.0] ],
"fmla.min_hours":  [ ["1993-08-05", 1250] ],
"ny.saww":          [ ["2025-01-01", 1757.19], ["2026-01-01", 1839.34] ]

```

Stage 3 — Fact input: what the user tells the system

The caller — an HR platform, a payroll system, a chatbot, or a person using the web checker — submits only facts about the situation. No legal knowledge is required to ask the question:

POST /determinations

```
{
  "facts": {
    "employee": {
      "work_state": "MN",
      "hire_date": "2025-03-01",
      "hours_last_12mo": 1400,
      "average_weekly_wage": 1100
    },
    "employer": { "total_employees": 85 },
    "event": { "type": "bonding", "start": "2026-09-01" }
  }
}
```

That is the entire interface: *where the person works, when they were hired, how much they work and earn, how big the employer is, and what is happening in their life.* The system — not the user — figures out which laws apply.

Stage 4 — Result: the determination, with its reasoning attached

The engine's actual response (trimmed for readability). Every conclusion points back to a Stage-1 citation, closing the loop:

```
{
  "as_of": "2026-09-01",
  "regimes": [
    {
      "name": "FMLA (federal)",
      "eligible": true,
      "findings": [
        { "description": "Employed by this employer for at least 12 months",
          "met": true, "citation": { "ref": "29 U.S.C. § 2611(2)(A)(i)" },
          "detail": "Tenure at leave start: 18.0 months" },
        { "description": "At least 1250 hours of service in the previous 12 months",
          "met": true, "citation": { "ref": "29 U.S.C. § 2611(2)(A)(ii)" },
          "detail": "Hours in previous 12 months: 1400" },
        { "description": "Employer has 50+ employees within 75 miles of the worksite",
          "met": true, "citation": { "ref": "29 U.S.C. § 2611(2)(B)(ii)" },
          "detail": "Employees within 75 miles: 85" }
      ],
      "entitlement": { "weeks": 12, "job_protected": true, "weekly_benefit": null,
        "notes": ["FMLA leave is unpaid; job protection and benefits continuation per
      ],
    },
    {
      "name": "Minnesota Paid Leave",
      "eligible": true,
      "findings": [
```

```

    { "description": "Base-period wages of at least 5.3% of the state average annu
      "met": true, "citation": { "ref": "Minn. Stat. § 268B.04" },
      "detail": "Base-period wages: $57,200" },
    { "description": "Employment in Minnesota is covered (nearly all employers, re
      "met": true, "citation": { "ref": "Minn. Stat. § 268B.01" },
      "detail": "Employer size 85; no minimum size applies." }
  ],
  "entitlement": { "weeks": 12, "job_protected": true, "weekly_benefit": 896.76,
    "notes": ["Combined family + medical leave is capped at 20 weeks per benefit y
  }
],
"interactions": [
  "FMLA and Minnesota Paid Leave cover the same qualifying reason: leave generally r
  "Per 2026 U.S. DOL guidance on FMLA/PFML interplay, an employer may not automatica
]
}

```

The \$896.76 is the Stage-2 formula applied to the Stage-3 facts under the Stage-2 parameters: 90% of the first \$711.50 (half the MN SAWW of \$1,423) is \$640.35, plus 66% of the remaining \$388.50 is \$256.41 — total **\$896.76 per week**. The answer isn't just a number; the arithmetic and its statutory source are reconstructible from the response.

Two variations that show the system's character

When the answer is no, you see exactly why. Same facts, but with only 900 hours worked: the FMLA determination flips to "eligible": false, and the justification tree shows one failed condition, with the other two still visibly satisfied:

```

{ "description": "At least 1250 hours of service in the previous 12 months",
  "met": false, "citation": { "ref": "29 U.S.C. § 2611(2)(A)(ii)" },
  "detail": "Hours in previous 12 months: 900" }

```

When the law requires judgment, the system says so instead of guessing. Change the leave reason to the employee's own health condition, and eligibility comes back "eligible": null — not yes, not no — because “serious health condition” is open-textured law that no rule engine should resolve:

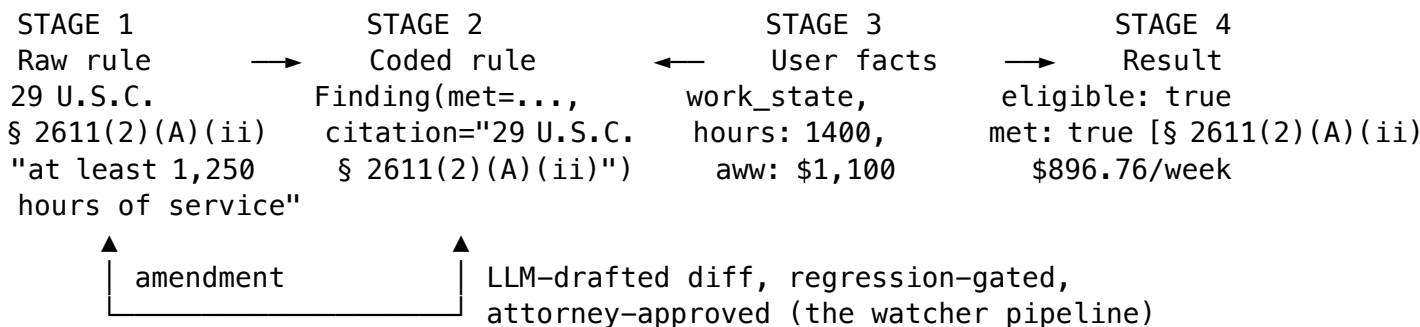
```

{ "description": "The condition qualifies as a 'serious health condition'",
  "met": null, "citation": { "ref": "29 C.F.R. § 825.113" },
  "detail": "Requires medical certification and case-by-case judgment; not determinabl

```

with "human_judgment": ["Whether the condition is a 'serious health condition' (29 C.F.R. § 825.113) must be resolved via medical certification."] — the question is routed to a person, with the governing regulation attached.

The chain, end to end



Each stage is linked to its neighbors: the coded rule cites the raw rule; the result cites the coded rule's citations; and when the raw rule changes, the amendment pipeline proposes the Stage-2 diff, the regression suite checks that history is preserved, and a human signs off. That closed loop — text to code to answer, with authority attached at every step — is what “Rules as Code” means in practice.

Prototype illustration. Statutory excerpts are abridged; parameter values are approximations pending counsel review. Decision support, not legal advice.